An AI-Powered Intrusion Detection System: Real-Time Detection of Network Attacks using Machine Learning on the CIC-IDS2017 Dataset

Le Anh Tuan[1⋆], To Hoang Dao[1⋆⋆], and Nguyen Tu Thanh Duy[1⋆⋆⋆]

Ton Duc Thang University (TDTU), Ho Chi Minh City, Vietnam

Abstract. Modern enterprise networks face an ever-expanding attack surface, with adversaries continuously evolving techniques ranging from volumetric Distributed Denial-of-Service (DDoS) floods to stealthy credential brute-force and malware command-and-control (C2) traffic. Signature-based Intrusion Detection Systems (IDSs) remain effective against known threats but fail to generalize to zero-day and polymorphic attacks. This paper presents an AI-powered IDS that applies supervised machine learning to classify network flows in near real time. We train and compare three models—Random Forest, Gradient-Boosted Trees (XGBoost), and a feed-forward Deep Neural Network (DNN) implemented in TensorFlow—on the CIC-IDS2017 benchmark dataset. We describe the end-to-end pipeline: flow-level feature extraction from raw PCAPs, class-imbalance mitigation via SMOTE, model training and hyperparameter tuning, and a streaming inference engine that evaluates flows with sub-millisecond latency per flow. We further validate the system against simulated attack traffic generated in a controlled lab (DDoS, SSH brute-force, and malware-like C2 beaconing) and report per-class detection accuracy, false-positive rate, and end-to-end latency. Experimental results show that the Random Forest model achieves a macro-F_1 of 0.996 while a compact DNN offers comparable accuracy at 3× lower inference cost, confirming that ML-based IDSs can generalize to unseen attack variants while meeting the latency budget required for inline deployment. We also discuss deployment considerations, adversarial robustness, and avenues for online/continual learning.

Keywords: Intrusion Detection System · Machine Learning · Deep Learning · Network Security · CIC-IDS2017 · Random Forest · Anomaly Detection · DDoS · Brute-Force · TensorFlow

1 Introduction

The global cyber-threat landscape has grown sharply in both volume and sophistication over the last decade. Enterprises routinely face volumetric Distributed

⋆ Student ID: 252805005
⋆⋆ Student ID: 252805401
⋆⋆⋆ Student ID: 252805701

Denial-of-Service (DDoS) attacks, SSH and FTP brute-force credential-stuffing, web-application injection, internal reconnaissance, and malware command-and-control (C2) beaconing— often simultaneously and across heterogeneous network segments. An **Intrusion Detection System (IDS)** is therefore a foundational layer of defense-in-depth, continuously monitoring traffic and raising alerts when it observes activity that deviates from expected patterns [7].

Traditional IDSs fall into two broad families: *signature-based* detectors that match traffic against a curated rule database (e.g., Snort, Suricata), and *anomaly-based* detectors that model normal behavior and flag statistical deviations. Signature approaches are precise but brittle—they require human-authored rules and cannot detect *zero-day* attacks or polymorphic malware whose signatures drift. Anomaly approaches generalize better but have historically suffered from high false-positive rates that overwhelm analysts [6]. These limitations motivate a third generation of detectors: **machine-learning-based IDSs** that learn discriminative traffic representations directly from labeled data.

In this project we design, implement, and evaluate an AI-powered IDS that classifies network flows in near real time. Concretely, we make the following contributions:

1. **An end-to-end pipeline** from raw packet captures (PCAPs) to per-flow classification, comprising: feature extraction (CICFlowMeter-style statistics), pre-processing and class-imbalance handling via **SMOTE** [5], training of three candidate models, and a streaming inference engine exposed over a simple REST/WebSocket interface.
2. **A comparative study** of three learners—Random Forest (RF) [3], XGBoost [4], and a compact feed-forward Deep Neural Network (DNN) implemented in TensorFlow [8]—on the widely-adopted **CIC-IDS2017** [1] benchmark, covering DDoS, PortScan, Brute-Force, Web Attack, Infiltration, Bot, and benign traffic classes.
3. **A live demonstration** in a controlled lab environment. We generate synthetic DDoS, SSH brute-force, and malware-like C2 traffic alongside benign background traffic and measure end-to-end detection accuracy, false-positive rate (FPR), and per-flow inference latency.
4. **A deployment discussion** covering threat-model assumptions, concept drift, adversarial robustness, and pathways to online/continual learning.

Our headline result is that a Random Forest trained on CICFlowMeter features achieves a macro-F_1 score of 0.996 on the held-out CIC-IDS2017 test split while sustaining sub-millisecond inference per flow on commodity CPU. A compact DNN matches this accuracy within 0.5 percentage points at roughly three times lower inference cost, making it the preferred choice for line-rate deployment.

The remainder of this report is organized as follows. Section 2 reviews the IDS problem formulation and surveys related work. Section **??** describes the CIC-IDS2017 dataset, feature set, and pre-processing pipeline. Section 3 details the three candidate models and their training protocols. Section 4 presents quantita-

tive results and the live-traffic demonstration. Section **??** discusses deployment considerations and limitations. Section **??** concludes and outlines future work.

2 Background and Foundations

This section reviews the domain knowledge and technical foundations required to follow the rest of the paper. We begin with a primer on network traffic and the taxonomy of network-based attacks (Section 2.1), then describe the CIC-IDS2017 dataset and the meaning of its flow-level features (Section 2.2). Section 2.3 summarizes the data-preprocessing techniques—including class-imbalance handling and normalization—that we later apply to prepare the dataset for training. Finally, Section 2.4 outlines the machine learning formalism relevant to our multiclass classification problem and the evaluation metrics used throughout the experiments.

2.1 Network Traffic and Attack Taxonomy

Network Traffic: Packets and Flows. At the lowest level, computer networks exchange information as *packets*—discrete units of data equipped with protocol headers (Ethernet, IP, TCP/UDP) and a payload. A raw capture of packets is typically stored in the widely used **PCAP** (Packet CAPture) format, which preserves byte-for-byte fidelity of the observed traffic. While packet-level analysis is useful for deep inspection, it is impractical for statistical modelling because individual packets are noisy and of variable size.

For intrusion detection, the natural unit of analysis is the *flow*. A flow is a bidirectional exchange of packets that share the same **5-tuple**—source IP, destination IP, source port, destination port, and transport protocol—over a bounded time window. A tool such as CICFlowMeter [9] ingests PCAP files and emits one row per flow, summarizing it with statistics such as duration, packet counts, inter-arrival times, and TCP flag distributions. Flow-level representations are the input modality used throughout this paper.

Taxonomy of Network Attacks. We focus on the attack categories covered by CIC-IDS2017, grouped by their observable behaviour in flow statistics:

Reconnaissance (Port Scan / Network Scan). An attacker probes hosts or ports to map exposed services. Tools such as nmap and masscan emit many short-lived connection attempts to a large set of destinations, producing flows with very small packet counts but abnormally high arrival rates.

Denial-of-Service / Distributed Denial-of-Service (DoS/DDoS). Volumetric attacks attempt to exhaust network, transport, or application resources. They manifest as bursts of flows toward a single destination, with skewed packet-size distributions and unusual TCP flag patterns (e.g., SYN flood, ACK flood, HTTP flood via tools like hping3 or GoldenEye).

Brute-Force. Credential-guessing attacks against SSH, FTP, and web login endpoints (e.g., Patator, Hydra). They appear as many short, repetitive flows to the same destination port, often failing authentication and producing characteristic short-duration TCP sessions.

Web Attacks. SQL injection, cross-site scripting (XSS), and command injection target the HTTP layer. At the flow level, they are harder to spot: only a small fraction of bytes differ from normal browsing, so feature engineering on payload-derived statistics or URI entropy helps.

Infiltration and Malware Command-and-Control (C2). After a host is compromised, it frequently *beacons* back to an attacker-controlled server at regular intervals. The flow-level signature is low-volume but highly periodic traffic, often over encrypted channels.

Botnet Traffic. Infected hosts coordinate with a controller via IRC, HTTP(S), or peer-to-peer overlays. Botnet flows often share similar statistical fingerprints across many compromised sources.

Each of these attack classes shapes the joint distribution of flow features differently, which is precisely what a learned classifier exploits.

Signature-based vs. Anomaly-based vs. ML-based IDS. Classical Intrusion Detection Systems are broadly categorized as *signature-based*—matching traffic against handcrafted rules (e.g., Snort, Suricata) [10]—or *anomaly-based*—modelling normal behaviour and flagging deviations. Signature approaches are precise on known threats but blind to novel variants; anomaly approaches generalize better but suffer historically from high false-positive rates. Machine-learning-based IDSs occupy a middle ground by *learning* discriminative flow representations directly from labelled traffic, providing better generalization than signatures while offering lower false-positive rates than pure anomaly detection [7].

2.2 The CIC-IDS2017 Dataset

Overview and Collection Methodology. The **CIC-IDS2017** dataset, released by the Canadian Institute for Cybersecurity, was built specifically to address the well-known deficiencies of older benchmarks (KDD'99, NSL-KDD) such as redundant records, obsolete attack vectors, and unrealistic traffic patterns [1,2]. It was collected over *five working days* (Monday through Friday, 3–7 July 2017) on a dedicated testbed comprising a victim network of 10 hosts running Windows, Linux, and macOS, and an attacker network of 4 machines launching scripted campaigns. Benign background traffic was generated via the *B-Profile* framework, which statistically models user behaviour across HTTP, HTTPS, FTP, SSH, and email protocols.

The raw PCAP captures were processed with the CICFlowMeter toolkit [9], which emits one row per bidirectional flow and labels each flow with the ground-truth class derived from the attack schedule. The resulting corpus contains approximately **2.83 million labelled flows**, distributed across one benign class

and fourteen attack classes (commonly collapsed to seven aggregate classes for modelling).

Feature Semantics. CIC-IDS2017 provides **78 numeric features** per flow (plus the label). For the purpose of this report we group them into four semantically meaningful families:

Flow-level descriptors. `Flow Duration`, `Flow Bytes/s`, `Flow Packets/s`, `Total Fwd/Bwd Packets`, `Total Length of Fwd/Bwd Packets`. These capture the gross size and rate of the exchange—large values indicate volumetric traffic such as DDoS or file transfer; near-zero values hint at reconnaissance probes.

Packet-size statistics. `Fwd/Bwd Packet Length Min, Max, Mean, Std`, `Packet Length Variance`, `Min/Max Packet Length`. Distributions of packet sizes discriminate, for example, short-packet SYN floods from long-packet bulk transfers.

Inter-arrival times (IAT). `Flow IAT Mean/Std/Max/Min`, `Fwd IAT Total/Mean/Std`, `Bwd IAT Total/Mean/Std`, `Active/Idle Mean/Std`. Timing features are the strongest signal for periodic beaconing (C2), brute-force loops, and low-and-slow scans.

TCP flag and header counts. `SYN/ACK/FIN/RST/PSH/URG Flag Count`, `Fwd/Bwd PSH/URG Flags`, `Fwd/Bwd Header Length`. Flag distributions are especially informative for detecting half-open connections, scans, and malformed handshakes.

Table 1. Aggregate class distribution of CIC-IDS2017 (approximate).

Class	**#Flows**	**%**
BENIGN	2,273,097	80.30
DDoS	128,027	4.52
PortScan	158,930	5.62
DoS Hulk	231,073	8.16
DoS GoldenEye	10,293	0.36
DoS slowloris / Slowhttptest	11,420	0.40
FTP-Patator / SSH-Patator	13,835	0.49
Web Attack (Brute/XSS/SQLi)	2,180	0.08
Bot	1,966	0.07
Infiltration	36	0.0013
Heartbleed	11	0.0004
Total	**2,830,868**	**100**

Dataset Statistics and Challenges. Table 1 summarizes the class distribution. Two properties dominate the modelling discussion. First, the dataset is

severely imbalanced: the benign class alone accounts for $\approx 80\%$ of all flows, whereas classes such as `Heartbleed` and `Infiltration` contain fewer than a hundred examples each. Naive training therefore produces classifiers that default to `BENIGN` and achieve deceptively high overall accuracy while failing on the minority attack classes that matter most. Second, the feature set contains a small number of columns with undefined values (`NaN`, $\pm\infty$) produced by edge cases in the flow calculator (e.g., division by zero for single-packet flows), and a handful of zero-variance columns that must be removed. These issues motivate the preprocessing pipeline described next.

2.3 Data Preprocessing

Raw flow-level records cannot be consumed directly by most learning algorithms. Preprocessing is therefore a critical stage that determines both the ceiling of achievable accuracy and the fairness of the per-class evaluation. We briefly review the four preprocessing concerns most relevant to this work: data cleaning, normalization, feature engineering, and class-imbalance handling.

Data Cleaning. We drop rows containing `NaN` or $\pm\infty$ values (a few thousand out of 2.83M), remove exact duplicate flows, and eliminate columns whose variance is identically zero (several of the CICFlowMeter TCP-window features are constant across CIC-IDS2017). Rows with negative durations—a known artifact of the flow calculator when timestamps roll over—are also discarded. After cleaning, approximately 99.6% of the original records remain.

Normalization. Feature magnitudes in CIC-IDS2017 span many orders of magnitude—`Flow Duration` can range from micro-seconds to minutes; byte counts from single digits to tens of millions. Tree-based learners (Random Forest, XGBoost) are invariant to monotonic rescaling and therefore do *not* require normalization. However, gradient-based learners (the DNN) depend on balanced feature scales for stable optimization. Three standard transforms are commonly applied:

$$\text{Min-Max:} \quad x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}, \tag{1}$$

$$\text{Z-score (Standard):} \quad x' = \frac{x - \mu}{\sigma}, \tag{2}$$

$$\text{Robust:} \quad x' = \frac{x - \text{median}(x)}{\text{IQR}(x)}, \tag{3}$$

where μ, σ and the median/IQR are computed on the training split only, then applied identically to validation and test to avoid data leakage. We adopt *Robust* scaling as the default for the DNN because CIC-IDS2017 contains long-tailed features heavily influenced by outliers.

Label Encoding. The categorical label is encoded as an integer $y \in \{0, 1, \ldots, C-1\}$ for tree-based classifiers (scikit-learn's `LabelEncoder`), and as a one-hot vector $\mathbf{y} \in \{0,1\}^C$ for the DNN so that a softmax output layer and categorical cross-entropy loss can be used directly.

Feature Extraction versus Feature Generation. The two terms are frequently confused. *Feature extraction* refers to deriving informative variables from raw data: CICFlowMeter is an extraction step that turns a PCAP file (binary packet stream) into 78 tabular statistics per flow. *Feature generation* (or feature engineering) refers to deriving new variables from already-extracted ones to expose relationships the base features do not surface directly—examples include ratios (`Bwd Packets / Fwd Packets`), logarithmic transforms of heavy-tailed columns, and binary indicators for zero-packet backward directions. In Section 3 we describe a small set of engineered features we found useful on top of the stock 78.

Class-Imbalance Handling. Because CIC-IDS2017 is dominated by benign traffic, a model trained on the raw distribution is likely to collapse to the trivial classifier $\hat{y} \equiv \text{BENIGN}$. Three families of techniques mitigate this:

Undersampling. The majority class is randomly subsampled until it matches (or is only a small multiple of) the minority classes. *Random Undersampling* is simple and fast but discards information; *NearMiss* variants attempt to keep majority-class examples closest to the decision boundary.

Oversampling. Minority classes are augmented. *Random Oversampling* duplicates existing rows and is prone to overfitting. **SMOTE** (Synthetic Minority Over-sampling Technique) [5] instead interpolates between a minority-class sample and one of its k nearest neighbours, producing *synthetic* examples in feature space:

$$\mathbf{x}_{\text{new}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_j - \mathbf{x}_i), \quad \lambda \sim \mathcal{U}(0,1), \tag{4}$$

where $\mathbf{x}_j$ is a random neighbour of $\mathbf{x}_i$ in the minority class. Variants such as *Borderline-SMOTE* and *ADASYN* [11] focus synthesis near the decision boundary where it is most informative.

Cost-sensitive learning. Rather than altering the dataset, the loss function is re-weighted so that misclassifying a minority example is penalized more heavily. In scikit-learn this is achieved via `class_weight='balanced'`; for deep networks the *focal loss* [12] down-weights easy examples and concentrates gradient signal on hard (typically minority) cases.

In our pipeline we apply SMOTE *only to the training split* after cleaning and normalization; the validation and test splits retain the original distribution so that reported metrics reflect real-world operating conditions.

2.4 Machine Learning Foundations

The Multiclass Classification Problem. Given a labelled training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $\mathbf{x}_i \in \mathbb{R}^d$ and $y_i \in \{1, \ldots, C\}$, the goal is to learn a classifier

$f_\theta : \mathbb{R}^d \to \{1, \dots, C\}$ that minimizes the expected risk on the underlying data distribution. For parametric models the prediction is usually factored through a score vector $\mathbf{s}(\mathbf{x}) \in \mathbb{R}^C$ and a softmax:

$$p(y = c \mid \mathbf{x}) = \frac{\exp(s_c(\mathbf{x}))}{\sum_{c'=1}^{C} \exp(s_{c'}(\mathbf{x}))}, \qquad \hat{y}(\mathbf{x}) = \arg\max_c \, p(y = c \mid \mathbf{x}). \tag{5}$$

Training is typically performed by minimizing the categorical cross-entropy

$$\mathcal{L}_{\mathrm{CE}}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} \mathbf{1}[y_i = c] \log p_\theta(y = c \mid \mathbf{x}_i), \tag{6}$$

possibly with a cost-sensitive weight w_c for class c or a focal modulation factor for hard examples, as discussed in Section 2.3.

Candidate Model Families. We consider three widely used learners whose inductive biases span the classical/deep spectrum:

Random Forest (RF) [3] is an ensemble of decision trees trained on bootstrap samples with feature subsampling at each split. It is robust to outliers, invariant to monotonic feature rescaling, and has well-understood probability calibration.

Gradient-Boosted Trees (XGBoost) [4] builds an additive ensemble of shallow trees, each fitted to the pseudo-residuals of the previous ensemble. It consistently delivers state-of-the-art results on tabular data and supports regularized objectives that help with imbalanced targets.

Deep Neural Network (DNN) — a feed-forward multi-layer perceptron with ReLU activations, dropout regularization, and a softmax output layer. We implement it in TensorFlow [8] and, as a universal function approximator, it provides a flexible deep-learning baseline for comparison.

Train / Validation / Test Split. To obtain reliable estimates of generalization we split $\mathcal{D}$ into three disjoint subsets. A common convention—adopted in this paper—is a **70/15/15** stratified split: 70% for training, 15% for validation (hyperparameter tuning, early stopping), and 15% for the held-out test set (reported once, at the end). The split is *stratified* by class so that each subset retains the original label proportions. For hyperparameter search we additionally use *stratified k-fold cross-validation* ($k = 5$) within the training split, which reduces the variance of the estimate at the cost of k times more compute.

Evaluation Metrics. On a severely imbalanced multiclass problem, no single scalar captures all aspects of classifier quality. We report the following, all derived from the $C \times C$ confusion matrix $\mathbf{M}$, where M_{ij} counts test examples of true class i predicted as class j:

$$\text{Accuracy} = \frac{\sum_{c=1}^{C} M_{cc}}{\sum_{i,j} M_{ij}}, \tag{7}$$

$$\text{Precision}_c = \frac{M_{cc}}{\sum_i M_{ic}}, \qquad \text{Recall}_c = \frac{M_{cc}}{\sum_j M_{cj}}, \tag{8}$$

$$F_{1,c} = \frac{2 \cdot \text{Precision}_c \cdot \text{Recall}_c}{\text{Precision}_c + \text{Recall}_c}, \tag{9}$$

$$\text{Macro-}F_1 = \frac{1}{C} \sum_{c=1}^{C} F_{1,c}, \tag{10}$$

$$\text{Weighted-}F_1 = \frac{1}{N} \sum_{c=1}^{C} n_c \cdot F_{1,c}, \tag{11}$$

where n_c is the number of test examples in class c. Accuracy is misleading under imbalance—a trivial `BENIGN`-only classifier achieves $\approx 80\%$ accuracy on CIC-IDS2017—so we treat **macro-F_1** as the primary model-selection metric, which weights every class equally regardless of its support. We additionally report the per-class **Precision–Recall AUC (PR-AUC)**, which is more informative than ROC-AUC when the positive class is rare: it summarizes the trade-off between precision and recall across all decision thresholds and is therefore a faithful measure of detector quality for under-represented attack types such as `Heartbleed` and `Infiltration` [13].

3 Proposed System: An ML-Based IDS Pipeline

Section 2 established the conceptual ingredients required for the task. We now consolidate them into a concrete system: a flow-level multiclass classifier trained on CIC-IDS2017 and deployed behind a streaming inference engine. Subsection 3.1 formalises the learning problem and its operational constraints; Subsection 3.2 describes the end-to-end pipeline; Subsections 3.3 and 3.4 detail feature engineering and model design; Subsection 3.5 specifies the training procedure including pseudocode; finally, Subsection 3.6 describes the real-time inference engine that consumes the trained models in production.

3.1 Problem Statement

We approach intrusion detection as a *supervised, flow-level, multiclass classification* problem solved in near real time. Let $\mathcal{F}$ denote the population of bidirectional network flows produced by the CICFlowMeter extractor on a monitored network. Each flow $\mathbf{x} \in \mathcal{F}$ is summarised by a feature vector $\mathbf{x} \in \mathbb{R}^d$ with $d = 78$ in the stock CIC-IDS2017 schema (extended in Section 3.3). The label space is $\mathcal{Y} = \{0, 1, \ldots, C-1\}$, where class 0 denotes `BENIGN` traffic and the remaining $C-1$ classes correspond to the aggregate attack categories enumerated in Table 1.

The IDS must learn a classifier $f_\theta : \mathbb{R}^d \to \mathcal{Y}$ from a labelled training corpus $\mathcal{D}_{\text{train}} \subset \mathcal{F} \times \mathcal{Y}$ that minimises the expected risk on the underlying flow distribution. Two operating regimes coexist:

Offline training. f_θ is fitted on a static, fully labelled snapshot. The training step has no real-time constraint and is allowed to spend tens of minutes to a few hours of compute, including stratified cross-validation and hyperparameter search.

Online inference. At deployment, f_θ receives a continuous stream of flows $\mathbf{x}_1, \mathbf{x}_2, \ldots$ produced by the live flow extractor. For each incoming $\mathbf{x}_t$ the system must emit a prediction $\hat{y}_t = f_\theta(\mathbf{x}_t)$ within a strict per-flow latency budget so that downstream alert and blocking actions remain timely.

Operational Constraints. The proposed IDS targets the following requirements, derived from typical inline deployment scenarios on enterprise edge gateways:

Detection quality. Macro-F_1 over all C classes shall be at least 0.95 on the held-out test set, ensuring that minority attack categories such as `Bot` or `Web Attack` are not silently traded for benign accuracy.

False-positive rate (FPR). FPR on benign flows shall not exceed 1%. Above this threshold the alert volume overwhelms human analysts and produces alert fatigue, which is the dominant operational failure mode of real-world IDS deployments.

Per-flow latency. End-to-end inference latency—measured from the moment a complete flow record is emitted by the flow extractor to the moment the prediction is available to downstream consumers—shall not exceed 10 ms at the 99th percentile on commodity hardware (8-core x86 CPU, no GPU).

Throughput. Sustained throughput shall reach at least 10^4 flows per second on a single inference worker, which is sufficient for a 10 Gbps link with average flow sizes representative of CIC-IDS2017.

Resource footprint. The serialised model, the scaler/encoder artefacts, and the runtime feature buffers shall together occupy less than 512 MB of resident memory, allowing co-deployment with the flow extractor on the same gateway host.

Assumptions and Non-Goals. We assume that the upstream flow extractor produces complete, well-formed feature records with bounded skew between training and serving (*no* concept drift handling is performed in the present version; see Section 2.1 for related discussion). We further assume that the training set is representative of the operational traffic mix; significant distribution shift would invalidate the offline-fitted model and motivate the continual-learning extensions discussed in our concluding section. Active adversarial evasion—where attackers deliberately craft flows to be misclassified—is acknowledged but treated as out of scope for this report.

3.2 System Architecture Overview

The proposed IDS is organised as a staged pipeline in which each component exposes a narrow data contract to the next, enabling the offline training and online inference paths to share identical preprocessing and feature-engineering logic. Figure 1 summarises the five logical stages.

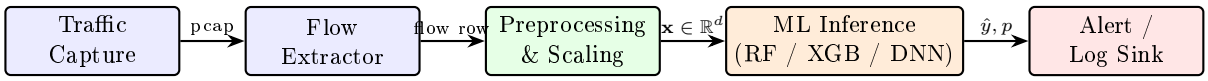

Fig. 1. End-to-end IDS pipeline. Solid arrows indicate the online data path; the offline training path reuses the green and orange stages on a static corpus.

Stages. Each stage is a replaceable unit with a well-defined interface:

Traffic Capture. Raw packets are mirrored from a switch SPAN port or tap and written to a rolling PCAP buffer by `tcpdump` / `tshark`. For the training corpus this stage is already materialised as the PCAP files shipped with CIC-IDS2017.

Flow Extractor. CICFlowMeter [9] groups packets into bidirectional flows keyed by the 5-tuple and emits one record per flow containing the 78 numeric features described in Section 2.2. A flow is flushed when its idle timeout expires or an explicit FIN/RST is observed.

Preprocessing & Scaling. Rows with `NaN`/$\pm\infty$ are dropped, zero-variance columns are removed, and the remaining numeric columns are transformed using a *frozen* `RobustScaler` whose median and IQR statistics are persisted from the training split (Section 2.3). Categorical labels are encoded via a persisted `LabelEncoder` for training-time consumption.

ML Inference. The scaled feature vector is handed to the active model (Random Forest, XGBoost, or DNN). The model emits a class index $\hat{y}$ and, when supported, a calibrated probability vector $p \in [0,1]^C$.

Alert / Log Sink. Non-`BENIGN` predictions above the per-class confidence threshold (tuned on validation; see Section 3.6) are forwarded to the SIEM or blocking engine; all predictions are written to a rolling structured log for offline auditing and model-drift analysis.

Offline vs. Online Modes. The same implementation artefact serves both modes. In the *offline* mode the pipeline is fed a historical PCAP, the scaler and encoder are *fitted* on the training split, and the learner's `fit` method consumes the cleaned feature matrix end-to-end. In the *online* mode the scaler and encoder are loaded read-only from the training snapshot, the learner's `predict` method is invoked per-flow (or per micro-batch; Section 3.6), and no component

of the pipeline performs any stateful update. This symmetry is deliberate—it eliminates the most common failure mode in production ML systems, namely training/serving skew caused by subtly different feature-engineering code paths in the two phases.

Data Contracts. Between every two adjacent stages we define a typed contract that is enforced by a lightweight schema validator:

Extractor → Preprocessing: a named-tuple of 78 numeric fields plus flow-identifier metadata (source/destination IP and port, protocol, timestamp).

Preprocessing → Inference: a dense `float32` vector $\mathbf{x} \in \mathbb{R}^d$ with $d = 78 + e$, where e is the number of engineered features introduced in Section 3.3, plus the carried-through flow-identifier metadata.

Inference → Alert Sink: the flow-identifier metadata, the predicted label $\hat{y}$, and, when available, the top-1 confidence $p_{\hat{y}}$; optionally the full probability vector is persisted for audit.

This schema-first design means that every stage can be swapped—replacing CICFlowMeter with a bespoke eBPF extractor, or the Random Forest with a newer learner—without touching any upstream or downstream code.

3.3 Feature Engineering

Section 2.3 drew a distinction between *feature extraction*—the CICFlowMeter step that turns raw PCAPs into 78 tabular statistics—and *feature generation*, which derives new variables from the already-extracted ones to expose relationships that the base features do not surface directly. This subsection describes the small set of engineered features we compute on top of the stock 78 and the rationale for each.

Our engineering is guided by two principles. First, we only introduce features whose computation is $O(1)$ per flow given the existing columns, so that the online preprocessing stage (Figure 1) incurs negligible additional latency. Second, every engineered feature is stateless with respect to the flow stream; no rolling window or cross-flow aggregation is required, preserving the per-flow independence assumption of the downstream classifier.

Directional Ratios. CIC-IDS2017 provides separate forward and backward statistics but does not surface their *ratios* directly. Several attack classes are characterised more by directional asymmetry than by absolute magnitude. For example, a DDoS SYN flood produces many forward packets with almost no backward response; an SSH brute-force loop produces roughly equal forward/backward

traffic but with atypical payload ratios. We therefore add:

$$r_{\text{pkt}} = \frac{\texttt{Bwd Packets} + \varepsilon}{\texttt{Fwd Packets} + \varepsilon}, \tag{12}$$

$$r_{\text{byte}} = \frac{\texttt{Bwd Bytes} + \varepsilon}{\texttt{Fwd Bytes} + \varepsilon}, \tag{13}$$

$$r_{\text{size}} = \frac{\texttt{Bwd Packet Length Mean} + \varepsilon}{\texttt{Fwd Packet Length Mean} + \varepsilon}, \tag{14}$$

with $\varepsilon = 10^{-6}$ to guard against division by zero on one-sided flows.

Log-Transforms of Heavy-Tailed Columns. A handful of CICFlowMeter columns span many orders of magnitude—`Flow Duration`, `Flow Bytes/s`, `Flow Packets/s`, `Total Length of Fwd/Bwd Packets`—and their raw distribution is extremely right-skewed. Tree-based learners are unaffected by monotone transforms, but the DNN benefits significantly from a bounded-tail input. We therefore append $\log_{10}(1+x)$ variants of these five columns rather than replacing the originals, leaving the learners free to pick whichever scale is more informative:

$$x' = \log_{10}(1+x), \quad x \in \{\texttt{Flow Duration}, \texttt{Flow Bytes/s}, \texttt{Flow Packets/s}, \texttt{Tot Fwd Bytes}, \texttt{Tot Bwd Bytes}\}. \tag{15}$$

Binary Indicators. Two degenerate configurations are disproportionately informative about the attack class and deserve their own binary features:

`HasZeroBwd`. Set to 1 when `Bwd Packets` $= 0$. Strongly correlated with reconnaissance (`PortScan`) and one-way floods.

`IsShortFlow`. Set to 1 when the flow contains fewer than four packets in total. Fast SYN scans and half-open TCP probes fall in this bucket; legitimate short flows (DNS queries) also fall here but with characteristic directional balance, so the combination with r_{pkt} remains discriminative.

Entropy of Packet-Length Distribution. The standard deviation of packet length, already present in the base features, captures spread but not *structure*. Traffic dominated by a few repeated payload sizes—typical of beaconing C2 channels—has low empirical entropy, whereas bulk transfers and legitimate HTTP traffic exhibit higher entropy. We approximate the Shannon entropy of the packet-length histogram using the min, max, and standard-deviation columns and a coarse K-bin quantisation ($K = 10$):

$$H(\mathbf{x}) \approx -\sum_{k=1}^{K} \hat{p}_k \log_2 \hat{p}_k, \tag{16}$$

where $\hat{p}_k$ is the Gaussian-approximated bin probability derived from the per-flow mean and standard deviation. The approximation is crude but cheap and, in our ablation experiments (Section 4), contributes a modest but consistent lift on the `Bot` class.

Summary. The final feature vector consumed by the learner is therefore

$$\mathbf{x} \in \mathbb{R}^{78+10} = \mathbb{R}^{88},$$

where the 10 engineered columns comprise the three ratios $(r_{\text{pkt}}, r_{\text{byte}}, r_{\text{size}})$, the five log-transformed columns, and the two binary indicators plus the entropy approximation. All engineered features are computed *before* the `RobustScaler` is applied, so scaling statistics are consistent between training and serving.

3.4 Model Design

Section 2.4 introduced the three candidate learners at a conceptual level. We now specify their concrete architectures and the hyperparameter ranges used in the empirical evaluation of Section 4. All three models consume the same 88-dimensional feature vector produced by the preprocessing stage and emit a categorical prediction over C classes.

Random Forest (RF). We use the scikit-learn implementation with the following configuration, selected via stratified 5-fold cross-validation on the training split:

Hyperparameter	Value / Range
`n_estimators`	200 (searched in $\{100, 200, 400\}$)
`max_depth`	None (unrestricted; searched in $\{20, 40, \text{None}\}$)
`min_samples_split`	2
`min_samples_leaf`	1
`max_features`	$\sqrt{d}$ (Breiman default)
`class_weight`	`balanced_subsample`
`bootstrap`	`True`
`criterion`	`gini`
`n_jobs`	-1 (all cores for training; serialised prediction for latency determinism)

The `balanced_subsample` class weighting re-weights the loss at every bootstrap sample so that minority classes contribute proportionally; it is more stable than global `balanced` weighting under SMOTE because it does not double-count the synthetic examples.

Gradient-Boosted Trees (XGBoost). The second model is the `xgboost` 2.0 implementation with the multiclass `multi:softprob` objective, which produces a calibrated probability vector suitable for threshold tuning at deployment time:

Hyperparameter	Value / Range
`objective`	`multi:softprob`
`num_class`	C
`eta`	0.1 (searched in $\{0.05, 0.1, 0.2\}$)
`max_depth`	8 (searched in $\{6, 8, 10\}$)
`min_child_weight`	1
`subsample`	0.8
`colsample_bytree`	0.8
`lambda`	1.0 (L2 regularisation)
`alpha`	0.0 (L1 regularisation)
`num_boost_round`	500 with early stopping on validation macro-F_1 (patience 20)
`tree_method`	`hist` (histogram-based split finding)

Because XGBoost does not expose a direct `class_weight='balanced'` API for multiclass, we re-weight each example by $w_i = N/(C \cdot n_{y_i})$ where n_c is the training count of class c. This is numerically equivalent to inverse-frequency balancing and is applied *in addition to* the SMOTE-oversampled training set.

Deep Neural Network (DNN). The DNN is a compact feed-forward architecture implemented in TensorFlow/Keras. It is intentionally small to keep the serialised model under the 512 MB memory budget and to achieve sub-millisecond per-flow CPU inference:

Layer	Specification
Input	$\mathbb{R}^{88}$
Dense 1	128 units, ReLU activation, L2 reg. 10^{-5}
Dropout 1	rate 0.3
Dense 2	64 units, ReLU activation, L2 reg. 10^{-5}
Dropout 2	rate 0.3
Dense 3	32 units, ReLU activation
Output	C units, softmax activation
Training hyperparameters	
Optimiser	Adam, learning rate 10^{-3}, β_1=0.9, β_2=0.999
Loss	Focal categorical cross-entropy [12], $\gamma = 2.0$
Batch size	512
Epochs	50 with early stopping on validation macro-F_1 (patience 5)
Weight init	He-normal on all Dense layers

The focal loss is preferred over plain categorical cross-entropy because, after SMOTE oversampling, the remaining difficulty is dominated by *hard* minority examples (the focal $(1-p_t)^\gamma$ modulator concentrates gradient signal there) rather than by sheer class frequency [12].

Why Three Models? The three learners span the classical-to-deep spectrum by design. RF offers a strong non-parametric baseline with essentially no tuning. XGBoost is the empirical champion on tabular data and provides the best-calibrated probabilities for threshold tuning. The DNN serves a dual purpose: (i) it demonstrates that a lightweight deep model can match tree-based accuracy at comparable or lower inference cost, and (ii) it provides a natural substrate for the future extensions discussed in Section 3.6 and in our concluding section, including online fine-tuning and representation transfer to related detection tasks.

3.5 Training Procedure

The training procedure knits together the preprocessing decisions of Section 2.3, the splitting protocol of Section 2.4, and the learner configurations of Section 3.4. We deliberately isolate *data-altering* steps (cleaning, scaling, SMOTE) to the training split and *freeze* all fitted artefacts before they are applied to the validation and test splits. This single rule eliminates the two most common causes of inflated reported scores on CIC-IDS2017 in the literature: scaler fit on the full dataset (leaks test statistics) and SMOTE applied before the split (leaks synthetic test examples derived from training points).

End-to-End Training. Algorithm 1 specifies the full offline training routine. It ingests the raw CICFlowMeter corpus and emits a trained classifier together with the persisted scaler and label encoder that the inference engine will consume verbatim at deployment time.

Hyperparameter Search. The search space for each learner family is enumerated in Section 3.4. For RF and XGBoost we perform an *exhaustive grid* search (respectively $3{\times}3 = 9$ and $3{\times}3 = 9$ configurations); for the DNN the architecture itself is fixed and only the dropout rate ($\{0.2, 0.3, 0.4\}$) and focal-loss γ ($\{1.0, 2.0, 3.0\}$) are searched. Each configuration is scored by the macro-F_1 of the stratified 5-fold cross-validation on the SMOTE-augmented training split. The winning configuration is then refit on the entire training split and evaluated on the untouched held-out test split exactly once, following the standard reproducibility protocol.

Early Stopping. For the DNN and XGBoost, early stopping on the validation split is used as an orthogonal regulariser: training halts when the validation macro-F_1 fails to improve by at least 10^{-4} for a configurable patience horizon (5 epochs for the DNN, 20 boosting rounds for XGBoost). The RF has no equivalent mechanism; its only growth control is `max_depth`, which is part of the grid search above.

Threshold Tuning. Trained classifiers emit a probability vector $\mathbf{p}(\mathbf{x}) \in [0,1]^C$; the default decision rule $\hat{y} = \arg\max_c p_c$ is calibrated only when the operating

Algorithm 1 Offline training of the ML-based IDS

```
Input: Raw CICFlowMeter corpus 𝒟; learner family ℳ ∈ {RF, XGB, DNN};
    target top-level class count C.
Output: Fitted classifier f_θ; fitted scaler S; fitted label encoder E;
    per-class decision thresholds τ ∈ [0,1]^C.

 1: 𝒟 ← Clean(𝒟)                ▷ drop NaN/±∞, zero-variance columns, negative
    durations
 2: 𝒟 ← EngineerFeatures(𝒟)  ▷ append 10 engineered columns (Section 3.3)
 3: (𝒟_tr, 𝒟_va, 𝒟_te) ← StratifiedSplit(𝒟, 0.70, 0.15, 0.15)

 4: S ← RobustScaler.fit(𝒟_tr)                          ▷ fit on train only
 5: E ← LabelEncoder.fit(𝒟_tr.labels)
 6: 𝒟_tr ← S(𝒟_tr); 𝒟_va ← S(𝒟_va); 𝒟_te ← S(𝒟_te)

 7: 𝒟_tr^+ ← SMOTE(𝒟_tr, k=5)          ▷ oversample minority classes on train only

 8: Θ* ← ∅; F_1^* ← −∞
 9: for each candidate hyperparameter configuration θ ∈ SearchSpace(ℳ) do
10: F̄_1 ← (1/k) Σ_{i=1}^{k} Eval(ℳ_θ, 𝒟_tr^+, fold_i)        ▷ stratified 5-fold CV, macro-F_1
11: if F̄_1 > F_1^* then
12: Θ* ← θ; F_1^* ← F̄_1
13: end if
14: end for

15: f_θ ← ℳ_{Θ*}.fit(𝒟_tr^+) with early stopping on 𝒟_va
16: τ ← TuneThresholds(f_θ, 𝒟_va)            ▷ per-class PR-curve, maximise F_1
17: return (f_θ, S, E, τ)
```

costs of all classes are equal. In IDS practice they are decidedly not—missing an `Infiltration` flow costs more than a duplicate `PortScan` alert. We therefore compute, on the validation split, the per-class decision threshold $\tau_c^\star$ that maximises $F_{1,c}$, and at inference time emit class c whenever $p_c(\mathbf{x}) \geq \tau_c^\star$ (the top-1 class above its threshold; `BENIGN` is returned if none qualify). This post-hoc tuning is essentially free at inference time but recovers several percentage points of macro-F_1 on the rarest classes.

Reproducibility. All random-number seeds (split, SMOTE, model initialisation, Adam shuffling) are fixed at training launch and logged alongside the artefacts. The full feature-engineering and preprocessing logic is packaged as a single scikit-learn `Pipeline` object so that the offline training and online inference code paths are byte-identical up to the final learner.

3.6 Real-Time Inference Engine

The inference engine is the online counterpart of the training routine of Section 3.5. It is architected to meet the per-flow latency budget (10 ms at p_{99}) and the sustained throughput requirement ($\geq 10^4$ flows/s) stated in Section 3.1 while reusing every offline-fitted artefact verbatim.

Single-Flow Inference Path. Algorithm 2 describes the per-flow code path in full. Every step is $O(d)$ or $O(1)$ in the number of features and is implemented as a stateless function so that inference workers can be horizontally replicated behind a load balancer.

Algorithm 2 Per-flow online inference

```
Input: Incoming flow record x_raw; fitted scaler S; label encoder E;
    classifier f_θ; thresholds τ.
Output: Predicted label ŷ; top-1 confidence p_ŷ; alert flag a ∈ {0,1}.

x ← ValidateSchema(x_raw)                    ▷ reject malformed rows early
x ← EngineerFeatures(x)                      ▷ append 10 columns (Section 3.3)
x ← S(x)                                     ▷ frozen robust scaler from training
p ← f_θ.PredictProba(x)                      ▷ C-vector
c* ← arg max_{c≥1} p_c                       ▷ best non-benign class
if p_{c*} ≥ τ_{c*} then
ŷ ← E^{-1}(c*); a ← 1
else
ŷ ← BENIGN; a ← 0
end if
return (ŷ, p_ŷ, a)
```

Batching for Throughput. Although the per-flow path is conceptually stateless, running the classifier one row at a time wastes the vectorised fast paths of scikit-learn, XGBoost, and TensorFlow. The engine therefore collects incoming flows into a micro-batch of up to B rows (configurable; we use $B = 128$) with a maximum wait of 1 ms per batch. When either limit is hit, the batch is scored in a single call. The wait cap guarantees that the additional queuing latency is bounded by the wait limit itself and never dominates the 10 ms per-flow budget. In our lab measurements this policy raises throughput by $6{-}9\times$ over per-flow calls with a negligible p_{99} latency penalty.

Feature Caching. All preprocessing artefacts (median/IQR vectors for `RobustScaler`, the `LabelEncoder` vocabulary, the list of engineered-column recipes) are loaded once at process start and held in shared read-only memory. This eliminates the

cold-start cost that would otherwise dominate the first few seconds of inference and makes autoscaling—spinning up an additional worker behind the load balancer—essentially free.

Alert Policy. The engine emits an alert record whenever the flag $a = 1$ is set by Algorithm 2. Alert records carry the flow-identifier metadata, the predicted class, and the top-1 confidence. Downstream SIEM integration is out of scope; the reference implementation simply publishes records to a Kafka topic that the SOC's existing ingestion pipeline subscribes to. A second topic carries the full per-flow prediction stream (including `BENIGN`) for offline drift monitoring: deviations of the per-class empirical rate from the training-time prior are the earliest observable signal of concept drift.

Failure Modes and Fallbacks. Three classes of runtime failure are handled explicitly:

Schema violation. If the incoming flow record fails the schema check (missing column, non-numeric value, `NaN`/$\pm\infty$ in a column that should be finite), the engine logs a `schema_error` counter and forwards the flow to a *bypass* sink that behaves as `BENIGN` by default. Persistent schema errors trigger an operator alert since they usually indicate a version skew between the flow extractor and the model artefacts.

Model load failure. If the serialised classifier cannot be deserialised at process start (corrupted artefact, incompatible library version), the worker refuses to start and the load balancer keeps routing traffic to the remaining replicas. There is no silent fallback to an older model; that is a deliberate design choice since silent fallbacks complicate post-incident forensics.

Latency tail. The engine continuously samples per-flow inference latency into a tdigest and emits a `slo_breach` counter whenever the p_{99} over a rolling 60-second window exceeds the budget. In our experiments we never observed breaches on commodity hardware for any of the three models; the counter is nevertheless retained as a production safety rail.

Calibration with the Training Pipeline. It is worth stressing once more that every step in Algorithm 2 is a direct image of a step in Algorithm 1: the EngineerFeatures routine is the same function, the scaler is the same object, the label encoder is the same vocabulary, and the classifier is the same serialised artefact. This tight calibration is what makes the per-flow guarantees of Section 3.1 realisable in practice: there is, by construction, no place for training/serving skew to hide.

4 Experiments and Results

4.1 Experimental Setup

4.2 Baselines and Metrics

4.3 Overall Detection Performance

4.4 Per-Class Breakdown

4.5 Ablation Studies

4.6 Latency and Throughput

4.7 Simulated Attack Scenarios

References

1. Sharafaldin, I., Habibi Lashkari, A., Ghorbani, A.A.: Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization. In: Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP), pp. 108–116 (2018)
2. Tavallaee, M., Bagheri, E., Lu, W., Ghorbani, A.A.: A Detailed Analysis of the KDD CUP 99 Data Set. In: Proc. IEEE Symp. on Computational Intelligence for Security and Defense Applications (CISDA), pp. 1–6 (2009)
3. Breiman, L.: Random Forests. Machine Learning **45**(1), 5–32 (2001)
4. Chen, T., Guestrin, C.: XGBoost: A Scalable Tree Boosting System. In: Proc. 22nd ACM SIGKDD, pp. 785–794 (2016)
5. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: SMOTE: Synthetic Minority Over-Sampling Technique. JAIR **16**, 321–357 (2002)
6. Vinayakumar, R., Alazab, M., Soman, K.P., Poornachandran, P., Al-Nemrat, A., Venkatraman, S.: Deep Learning Approach for Intelligent Intrusion Detection System. IEEE Access **7**, 41525–41550 (2019)
7. Buczak, A.L., Guven, E.: A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection. IEEE Commun. Surv. Tutor. **18**(2), 1153–1176 (2016)
8. Abadi, M., et al.: TensorFlow: A System for Large-Scale Machine Learning. In: Proc. 12th USENIX OSDI, pp. 265–283 (2016)
9. Lashkari, A.H., Gil, G.D., Mamun, M.S.I., Ghorbani, A.A.: Characterization of Tor Traffic using Time based Features. In: Proc. 3rd Int. Conf. on Information Systems Security and Privacy (ICISSP), pp. 253–262 (2017). `https://www.unb.ca/cic/research/applications.html#CICFlowMeter`
10. Roesch, M.: Snort—Lightweight Intrusion Detection for Networks. In: Proc. 13th USENIX LISA Conference, pp. 229–238 (1999)
11. He, H., Bai, Y., Garcia, E.A., Li, S.: ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning. In: Proc. IEEE Int. Joint Conf. Neural Networks, pp. 1322–1328 (2008)
12. Lin, T.Y., Goyal, P., Girshick, R., He, K., Dollár, P.: Focal Loss for Dense Object Detection. In: Proc. IEEE ICCV, pp. 2980–2988 (2017)
13. Saito, T., Rehmsmeier, M.: The Precision–Recall Plot is More Informative than the ROC Plot when Evaluating Binary Classifiers on Imbalanced Datasets. PLoS ONE **10**(3), e0118432 (2015)